

THE PROMPT LADDER

A Progressive Prompt-Engineering Exercise

Track	General AI Fluency
Assignment	The Prompt Ladder

Overview

This exercise traces a single request — building a React settings page — through seven successive iterations. Each version adds one deliberate layer to the prompt (goal clarity, audience, context, format, and constraints), and each is paired with the model's output and an honest assessment of what improved and what still fell short. The final entry combines every effective layer into a single, production-ready prompt.

1 Version 0 — Baseline

PROMPT

Build a React settings page.

OUTPUT

The AI generated a React `SettingsPage` component that includes:

- State management using `useState`
- Sidebar navigation with Profile, Notifications, and Appearance tabs
- Editable Username and Email fields
- Notification and Dark Mode toggle switches
- A "Save Changes" button with a save handler
- Inline styling for layout and appearance

REPRESENTATIVE CODE EXCERPT

```
import React, { useState } from 'react';

const SettingsPage = () => {
  const [activeTab, setActiveTab] = useState('profile');
  const [settings, setSettings] = useState({
    username: 'JohnDoe',
    email: 'john@example.com',
    notifications: true,
    darkMode: false,
  });

  const handleSave = () => {
    console.log('Saving settings:', settings);
    alert('Settings saved successfully!');
  };
};
```

NOTES

What changed in the prompt?

No changes. This is my original weak prompt.

What actually improved in the output?

The AI generated a functional React settings page with multiple sections, state management using `useState`, editable form fields, and a Save button.

What still failed?

The output was too generic. It did not know what application the settings page was for, who the users were, or what technologies and design requirements should be used. It also used inline styles instead of a scalable styling approach.

What would I try next?

I will make the goal clearer so the AI understands what kind of settings page I need.

2 Version 1 — Clearer Goal

LAYER ADDED: CLEARER GOAL

PROMPT

Build a React settings page for an AI-powered application where users can manage their account preferences and application settings.

OUTPUT

The AI generated an AI-focused settings page with:

- Model configuration
- API key management
- Temperature controls
- Auto-save preferences
- AI-related configuration sections

REPRESENTATIVE CODE EXCERPT

```
import React, { useState } from 'react';

const AISettings = () => {
  const [activeTab, setActiveTab] = useState('account');
  const [config, setConfig] = useState({
    model: 'gpt-4o',
    temperature: 0.7,
    apiKey: 'sk-...',
    contextWindow: 8192,
    autoSave: true
  });

  const handleUpdate = (e) => {
    const { name, value, type, checked } = e.target;
    setConfig(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };
};
```

NOTES

What changed in the prompt?

I added a clearer goal by specifying that the settings page is for an AI-powered application.

What actually improved in the output?

The output became much more relevant to an AI application. Instead of generic profile settings, it included AI-specific features such as model selection, API key management, temperature settings, and system preferences.

What still failed?

The AI still made assumptions about the application's users and implementation. It used inline styles and did not follow any specific design system or project requirements.

What would I try next?

I will define the target audience so the AI can tailor the interface to the users' needs.

3 Version 2 — Defined Audience

LAYER ADDED: DEFINED AUDIENCE

PROMPT

Build a React settings page for an AI-powered application where users can manage their account preferences and application settings.

The target audience is frontend developers who use AI tools in their daily workflow.

OUTPUT

The AI generated a settings page designed for frontend developers that includes:

- AI model selection
- API key management
- Developer preferences
- Theme and notification settings
- Configuration options relevant to AI development

REPRESENTATIVE CODE EXCERPT

```
import React, { useState } from 'react';

// Reusable Input Component
const SettingField = ({ label, children, description }) => (
  <div style={{ marginBottom: '1.5rem' }}>
    <label>{label}</label>
    {children}
  </div>
);

const AISettings = () => {
  const [settings, setSettings] = useState({
    model: 'gpt-4o',
    temperature: 0.7,
    streamResponse: true,
    customSystemPrompt: 'You are an expert React developer.',
    theme: 'dark'
  });
```

NOTES

What changed in the prompt?

I added the target audience by specifying that the settings page is intended for frontend developers who use AI tools.

What actually improved in the output?

The output became more tailored to frontend developers. It introduced developer-specific features such as a reusable component, custom system prompts, token streaming, and AI model configuration instead of only generic account settings.

What still failed?

The response still assumed the project's styling and architecture. It continued using inline styles and did not follow any specific project or design system.

What would I try next?

I will add real project context so the AI can better understand the application's purpose and generate a more relevant solution.

4 Version 3 — Real Context

LAYER ADDED: REAL CONTEXT

PROMPT

Build a React settings page for FlyRank AI, an AI-powered application where users can manage their account preferences and application settings.

The target audience is frontend developers who use AI tools in their daily workflow.

FlyRank AI helps developers generate, review, and improve frontend code using AI. The settings page should allow users to configure AI models, response behavior, code generation preferences, and developer workflow options.

OUTPUT

The AI generated a FlyRank AI-specific settings page that includes:

- FlyRank AI branding
- Code generation preferences
- Framework selection (React, Next.js, Vue)
- AI model configuration
- Workflow integration settings
- Custom system prompt
- Auto-linting and developer workflow options

REPRESENTATIVE CODE EXCERPT

```
import React, { useState } from 'react';

const FlyRankSettings = () => {
  const [config, setConfig] = useState({
    model: 'claude-3-5-sonnet',
    framework: 'react-typescript',
    linting: true,
    autoFormat: true,
    systemPrompt:
      'You are a Senior Frontend Engineer. Prioritize type safety, accessibility, and modular components.',
    streamResponse: true
  });

  const handleUpdate = (key, value) =>
    setConfig(prev => ({ ...prev, [key]: value }));
```

NOTES

What changed in the prompt?

I added real project context by explaining what FlyRank AI does and what the settings page should support.

What actually improved in the output?

The output became much more project-specific. It generated settings directly related to FlyRank AI, such as framework selection, code generation preferences, workflow integrations, and AI model configuration instead of generic AI settings.

What still failed?

The response still assumed the UI implementation and styling approach. It continued using inline styles and did not organize the response into separate planning and implementation sections.

What would I try next?

I will specify the required output format so the AI presents the solution in a clearer and more structured way.

5 Version 4 — Specified Output Format

LAYER ADDED: SPECIFIED OUTPUT FORMAT

PROMPT

Build a React settings page for FlyRank AI, an AI-powered application where users can manage their account preferences and application settings.

The target audience is frontend developers who use AI tools in their daily workflow.

FlyRank AI helps developers generate, review, and improve frontend code using AI. The settings page should allow users to configure AI models, response behavior, code generation preferences, and developer workflow options.

Present your response in the following format:

1. Component overview 2. React component code 3. State management explanation 4. Suggestions for future improvements

OUTPUT

The AI organized its response into clearly defined sections instead of returning only code. It included:

- Component overview
- Complete React component
- State management explanation
- Suggestions for future improvements

REPRESENTATIVE CODE EXCERPT

```
import React, { useState } from 'react';

const FlyRankSettings = () => {
  const [settings, setSettings] = useState({
    model: 'claude-3-5-sonnet',
    temperature: 0.2,
    stream: true,
    framework: 'react-typescript',
```

```
    linting: true,
    systemPrompt:
      'You are a senior frontend engineer. Output clean, modular, and type-safe code.'
  });

  const updateSetting = (key, value) =>
    setSettings(prev => ({ ...prev, [key]: value }));
```

NOTES

What changed in the prompt?

I specified the output format by asking the AI to organize its response into a component overview, React component code, state management explanation, and future improvement suggestions.

What actually improved in the output?

The response became much better organized. Besides generating the React component, it explained the component structure, state management approach, and suggested future enhancements, making the output easier to understand and implement.

What still failed?

The implementation still relied on inline styles and made assumptions about coding standards. It did not follow any explicit implementation constraints such as using Tailwind CSS or accessibility guidelines.

What would I try next?

I will add implementation constraints so the AI follows specific project standards and produces code that better matches the intended architecture.

6 Version 5 — Constraints

LAYER ADDED: CONSTRAINTS

PROMPT

Build a React settings page for FlyRank AI, an AI-powered application where users can manage their account preferences and application settings.

The target audience is frontend developers who use AI tools in their daily workflow.

FlyRank AI helps developers generate, review, and improve frontend code using AI. The settings page should allow users to configure AI models, response behavior, code generation preferences, and developer workflow options.

Present your response in the following format: 1. Component overview 2. React component code 3. State management explanation 4. Suggestions for future improvements

Constraints: Use functional React components and Hooks only · Use Tailwind CSS instead of inline styles · Keep the component modular and reusable · Follow accessibility best practices (ARIA labels and keyboard navigation) · Do not use external UI libraries.

OUTPUT

The AI generated a React settings page that followed all specified constraints:

- Tailwind CSS instead of inline styles
- Functional components with Hooks

- Accessible form elements using htmlFor and ARIA attributes
- Modular and reusable component structure
- No external UI libraries

REPRESENTATIVE CODE EXCERPT

```
import React, { useState } from 'react';

const FlyRankSettings = () => {
  const [activeTab, setActiveTab] = useState('models');
  const [settings, setSettings] = useState({
    model: 'claude-3-5-sonnet',
    framework: 'react-typescript',
    linting: true,
  });

  return (
    <div className="max-w-4xl mx-auto p-6 bg-white min-h-screen">
      <nav aria-label="Settings navigation">
        {/* Navigation buttons */}
      </nav>
    </div>
  );
};
```

NOTES

What changed in the prompt?

I added implementation constraints, including the use of Tailwind CSS, functional React components with Hooks, accessibility best practices, modular components, and no external UI libraries.

What actually improved in the output?

The generated code followed the requested technical standards. It replaced inline styles with Tailwind CSS, included accessibility features such as ARIA attributes and proper labels, and produced a cleaner, more maintainable implementation.

What still failed?

Adding several constraints made the response longer and more detailed. Although the code quality improved, the AI had less flexibility in how it approached the solution, making the output more complex to read. This is my honest "this didn't help" observation: the additional constraints improved consistency but also increased the complexity and length of the response.

What would I try next?

I would add verification requirements so the AI checks that the code meets accessibility, responsiveness, and project standards before presenting the final solution.

7 Final Best Prompt

The version below combines every layer that proved effective across the ladder — clear goal, defined audience, real project context, structured output format, and explicit technical constraints.

Build a React settings page for FlyRank AI, an AI-powered application where frontend developers can manage their account preferences and AI configuration.

Context:

FlyRank AI helps developers generate, review, and improve frontend code using AI. The settings page should include options for AI model selection, code generation preferences, workflow configuration, and developer-specific settings.

Present your response in the following format:

- 1. Component overview*
- 2. React component code*
- 3. State management explanation*
- 4. Suggestions for future improvements*

Requirements:

- Use functional React components and React Hooks only.*
- Use Tailwind CSS instead of inline styles.*
- Build reusable and modular components.*
- Follow accessibility best practices, including proper labels, ARIA attributes, and keyboard navigation.*
- Do not use external UI libraries.*